Agentproof: A Real-World Base-Rate Study of Static Verification for Agent Workflow Graphs

Abstract

Agent frameworks increasingly encode tool-using behavior as explicit workflow graphs whose topology can be analyzed before deployment—yet whether such graphs actually contain the structural defects static verification targets has never been measured. We build AGENTPROOF, which automatically extracts a unified abstract graph model from four agent frameworks (LangGraph, CrewAI, AutoGen, Google ADK) with no hand-written model, applies six structural checks with witness traces, and compiles a temporal policy DSL (a safety fragment of LTL) to DFAs evaluated both statically, via a graph $\times$ DFA product, and at runtime. Used as a measurement instrument, it enables the first base-rate study of topology defects in real agent workflows: 930 mined from 253 GitHub repositories across all four frameworks, a 120-workflow sample validated against independent LLM-reconstructed ground-truth graphs and adversarial triage. The result is negative: real workflows are tiny (median two non-sentinel nodes) static pipelines, and of 187 flagged defects only two are genuine—43% are extraction artifacts, 50% intentional design. We diagnose extraction fidelity (edge recall 0.64 on real code versus near-perfect on stubs, and as low as 0.37 for implicit-topology frameworks like ADK), not the check algorithms, as the binding constraint, and give the static machinery a defect-independent job: the product construction proves which runtime monitors can never fire, so they can be soundly pruned.

1 Introduction

Large language models are increasingly deployed as autonomous agents that call external tools and APIs [19, 22]. Several orchestration frameworks structure this behavior as explicit *workflow graphs*: LangGraph [3], CrewAI [1], AutoGen [21], and Google ADK [2] each represent computation steps as nodes and allowable transitions as edges. Yet safety enforcement in these systems remains almost entirely a *runtime* concern: tools such as NeMo Guardrails [16] and Llama-Guard [12] filter individual calls at execution time, detecting a violation only when the offending path is exercised.

Some defects, however, are topological. Consider an email-triage agent whose developer adds a `draft_response` node but forgets to connect it to `send`: normal-priority emails silently halt at a dead end. A content-level runtime guardrail cannot catch this unless that path runs, but a static analysis of the graph flags the dead end and emits a witness trace. In principle classical model checkers [6, 7, 11] verify such properties, but they require hand-translating the system into Promela or SMV—prohibitive for developers iterating on workflows.

This raises a question nobody has answered empirically: *do real agent workflows actually contain the structural defects that static verification targets?* We build AGENTPROOF to find out. It extracts a unified graph model directly from four frameworks' APIs (no manual modeling), runs six structural checks with witness traces, and compiles a temporal policy DSL to DFAs checked statically via a graph $\times$ DFA product and at runtime. We then use it as a *measurement instrument* on 930 workflows mined from 253 public GitHub repositories.

The answer is deflationary, and that is the point. Topology defects are rare; what limits static verification in practice is not the check algorithms but *extraction fidelity*, which we quantify and which turns out to be strongly framework-dependent. The static machinery still earns a defect-independent job: proving which runtime monitors can never fire, so they can be soundly pruned.

Contributions.

- **The first real-world base-rate study** of structural (topology) defects in agent workflows: 930 workflows from 253 public GitHub repositories across all four frameworks, with a 120-workflow subsample validated against independent LLM-reconstructed ground-truth graphs and adversarial triage (Section 3). Because nobody had measured it, the rarity is itself the contribution: the validated topology-defect rate is $\approx 0\%$ (2 of 187 flags genuine; 43% extraction artifacts, 50% intentional design), and real workflows are mostly small static pipelines (median two non-sentinel nodes).
- **AGENTPROOF**, the measurement instrument: automatic extraction of a unified graph model from four heterogeneous frameworks with no hand-written formal model, six structural checks with witness traces, and a seven-form temporal DSL over the safety fragment of LTL compiled to DFAs and evaluated statically and at runtime (Section 2).
- **A quantified diagnosis**: extraction fidelity, not the checks, is the binding constraint. Edge recall drops to 0.64 on real code (0.37 on ADK, 0.77 on AutoGen) and node-kind accuracy to 0.83, both near-perfect on stubs; every triaged structural flag is an extraction artifact, and such flags arise only on LangGraph because the other extractors impose connected topologies by construction.

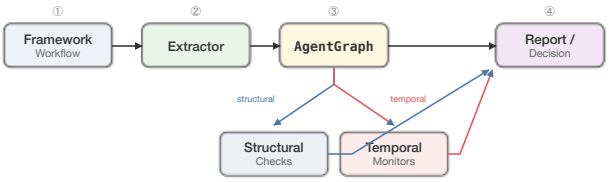

Figure 1: Agentproof extracts a unified graph model from four frameworks, runs structural checks and DFA-compiled temporal policies statically (and at runtime), and emits witness traces.

- **A defect-independent job** for the static machinery: the graph × DFA product proves which policies can never reach a violation, so their runtime monitors are provably inert and can be pruned (a mean 14 of 15 per workflow; Section 4), positioning static verification as a sound monitor-selection layer that bridges to leaner runtime shields.

2 The Agentproof System

Unified graph model. Agentproof represents a workflow as a directed graph $G = (V, E, v_0, V_f)$ with a distinguished entry v_0 and exit set V_f. Each node carries a kind $\kappa_V(v) \in \{\text{ENTRY}, \text{EXIT}, \text{TOOL}, \text{LLM}, \text{ROUTER}, \text{HUMAN}, \text{SUBGRAPH}\}$ and, for tool nodes, a set of bound tool names; each edge is DIRECT, CONDITIONAL, PARALLEL, or LOOP. Framework-specific extractors bridge four heterogeneous representations—each with different entry/exit conventions, edge semantics, and hierarchy models—into this single typed model, eliminating the manual translation that SPIN or NuSMV require. Extraction has two paths: a runtime extractor that reads the compiled graph object, and a static AST fallback (used for large-scale mining, where installing every repository's dependencies is infeasible).

Structural checks. Six checks run over the topology, each in $O(|V| + |E|)$ and each emitting a *witness trace* on failure. *Exit reachability* verifies $V_f \subseteq \text{Reach}(v_0)$; *reverse reachability* flags livelock nodes reachable from v_0 that cannot reach any exit; *dead-end detection* flags non-exit nodes with no successor; *router shape* requires router outgoing edges to be conditional; *human-gate coverage* checks that no path from v_0 to a sensitive tool avoids every HUMAN node; and *tool declaration* flags tool nodes with an empty tool set. Witness traces follow standard model-checking practice [7] and pinpoint the offending path.

Temporal policies. Beyond topology, Agentproof supports a policy DSL covering the safety fragment of LTL [17], compiled to deterministic finite automata. The DSL has seven forms: forbidden (`G ! a`), implication-future (`a -> F b`), until, bounded response (`a -> F [<=k] b`), response chain, conjunction, and disjunction. Each node emits an event symbol $\sigma(v)$ from its kind and tool bindings; base patterns compile to 2–3 state DFAs and bounded response to $k+2$ states (full grammar in the appendix).

Table 1: AST-extractor fidelity on real workflows (n=120). Perfect on stubs; on real code fidelity is lower and strongly framework-dependent.

Framework	n	Edge P	Edge R	Kind acc
LangGraph	40	0.956	0.682	0.647
CrewAI	20	0.700	0.692	0.929
AutoGen	35	0.957	0.770	0.900
ADK	25	0.403	0.367	0.943
Overall	**120**	**0.798**	**0.644**	**0.829**

Static temporal verification. Policies are checked *statically* via a graph × DFA product: BFS from (v_0, q_0) advances the DFA at each node and follows graph edges, and a reachable product state in the DFA violation set witnesses a violation. Because the product explores a superset of a workflow's runtime paths, if no violation state is reachable the property holds on *every* runtime execution—a sound guarantee we exploit in Section 4. Explored states are bounded by $|V| \cdot |Q|$, so verification is linear in graph size for fixed-size DFAs; the same monitors also run at runtime over an event stream with $O(1)$ per-event overhead.

3 Real-World Base-Rate Study

To estimate defect *base rates* on independently authored workflows—and to measure extractor fidelity on code we did not write—we mined public GitHub repositories across all four frameworks.

Mining. Using GitHub code search we streamed each candidate repository through clone, extraction, and cleanup, yielding 930 workflow graphs from 253 distinct repositories (LangGraph 402, AutoGen 361, CrewAI 114, ADK 53). Extraction used the static AST fallback. Mined workflows are small: a median of 2 non-sentinel nodes (mean 3.9), predominantly static linear pipelines.

Validation. On a stratified 120-workflow sample (40 LangGraph, 20 CrewAI, 35 AutoGen, 25 ADK) we ran two independent LLM-agent passes. First, an agent reconstructed a ground-truth graph from source alone (blind to the extractor, following its node-id conventions), against which we scored node/edge precision–recall and node-kind accuracy. Second, for each of the 187 flagged defects one agent labelled it REAL, EXTRACTION-ARTIFACT, or INTENTIONAL from source, and an adversarial verifier attempted to overturn each label (disagreements recorded as ARGUABLE).

Table 2: Adversarial triage of 187 flagged defects. Every structural flag is an extraction artifact; the human-gate flag is predominantly intentional.

Check	Real	Artifact	Intent.	Arg.
exit_reachability	0	14	0	0
reverse_reachability	0	25	0	0
dead_ends	0	24	0	1
router_shape	0	3	0	0
tool_declarations	0	5	0	0
human_presence	2	9	93	11
Total	**2**	**80**	**93**	**12**

Extraction fidelity is the bottleneck. Node detection stays strong (overall P=0.91, R=0.85), but edge recall falls to 0.644 and node-kind accuracy to 0.829, ranging from 0.37 edge recall on ADK to 0.77 on AutoGen (Table 1). The frameworks that encode topology *implicitly*—ADK's nested `SequentialAgent`/ `ParallelAgent` composition, AutoGen's participant lists with dynamic speaker selection—are hardest: the extractor must guess an edge set the source never states, and for ADK it is wrong more often than right. On LangGraph, whose edges are explicit, the dominant error is instead node-kind (`tool`→`llm`: nodes invoking tools in their body with no declared binding).

Defects are rare. Of 187 flagged defects, only 2 (1.1%) are genuine bugs—both missing human gates—while 43% are extraction artifacts and 50% intentional design (Table 2). *Every one of the 71 structural flags is an extraction artifact*, and structural flags are entirely a LangGraph phenomenon: in the raw study they fire on 78% of LangGraph workflows but 0% of AutoGen, CrewAI, and ADK, whose extractors build a connected topology by construction and cannot emit a dead end or unreachable exit. The structural signal is thus an artifact of the one extractor that recovers real edges, not a property of the workflows.

Risk-aware gating. The human-gate check fires on 91% of workflows, but only 2 of 115 firings are genuine—the rest are intentional low-risk tasks. A risk-aware variant flags a workflow only when a *sensitive* tool (destructive/write/exec/communication/ financial, by a keyword lexicon) is reachable without a HUMAN gate, reusing the sound coverage check of Section 2. On the 930 mined workflows it fires on *zero*: none declares a sensitive tool, as they are read-only retrieval pipelines that need no review. But its recall is itself bounded by extraction fidelity—an in-body sensitive call with no declared binding (the `tool`→`llm` error above) is invisible—so a sound sensitivity signal depends on the same higher-fidelity extraction we identify as the binding constraint.

4 Controlled Evaluation and Monitor Selection

Controlled detection test. A curated corpus of 18 workflows spanning all four frameworks, seeded with known anti-patterns, confirms the checks fire when defects are genuinely present: Agentproof detects every injected structural defect and, with a risk-aware sensitive-tool set, flags 8 workflows whose sensitive path (e.g. `account_create`, `publish`, `auto_remediate`) is reachable without a human gate, versus 10 for the blunt human-presence check—declining to flag two workflows that lack a human node but perform no sensitive action. This corpus is a controlled test that the algorithms work, not evidence of prevalence. Verification is sub-second even for synthetic graphs of 5,000 nodes (far larger than any observed workflow); details, per-framework extractor accuracy on stubs, and a Promela/SMV modeling-effort comparison are in the appendix.

Monitor selection via static pruning. The graph $\times$ DFA product gives the static machinery a job that does not depend on defects existing. Because the product explores a superset of runtime paths, a policy whose product reaches no violation state can never fire at runtime on that workflow: its monitor is provably inert and need not be deployed. This is a sound *monitor-selection* step—deploy per workflow only the monitors that can actually fire. Across the curated corpus and 15 policies, 252 of 270 monitor instances (93.3%) are provably inert, a mean of 14 of 15 monitors skipped per workflow, so runtime enforcement evaluates only the remaining 6.7%; multi-tool nodes are first split into single-tool chains so the analysis never under-approximates a node's events.

We report two honest limits. First, on today's small, mostly linear workflows almost all inertness is *alphabet-level* (the policy's atoms never appear); the reachability product proves only one further case, and its advantage over a plain symbol-set check grows with branching structure. Second, the fixed policy vocabulary does not match real code—none of the 930 mined workflows invokes any of the 15 policies' tools—so measuring pruning on real workflows requires instantiating policy templates against each workflow's own tools, which we leave to future work. Static monitor selection is thus a sound, cheap optimization whose leverage today is predominantly alphabet-level, and it is the concrete bridge to runtime enforcement.

5 Related Work

Runtime agent safety. NeMo Guardrails [16], Guardrails AI [10], LlamaGuard [12], and Rebuff [18] enforce safety at the point of LLM output or tool call, catching content-level violations only when the offending path runs. Agentproof is complementary: it verifies structural properties of the topology before deployment, and its monitor-selection layer (Section 4) decides which of these runtime monitors are even needed.

Model checking and workflow soundness. Classical model checkers—SPIN [11], NuSMV [6], CBMC [14]—verify the same class of properties but require manual translation into Promela/SMV/annotated C; Agentproof extracts the model automatically. Business-process soundness for Petri nets [20] and BPMN [8] studies analogous reachability properties, but targets visual notations rather than the programmatic API objects agent frameworks expose. Runtime-verification frameworks such as JavaMOP [13] and three-valued LTL monitoring [5] inform our lightweight DFA monitors.

Agent safety and evaluation. Work on agent safety emphasizes alignment, capability evaluation, and governance [4, 9, 15]. Our contribution is orthogonal and empirical: the first base-rate measurement of a specific, checkable defect class in deployed workflows, and a diagnosis of what actually limits static verification for them.

6 Conclusion

Agent frameworks encode behavior as explicit workflow graphs, and AGENTPROOF turns that structure into an analyzable, cross-framework model with six structural checks, witness traces, and a safety-fragment-LTL policy DSL evaluated both statically (via a graph $\times$ DFA product) and at runtime. Used as a measurement instrument on 930 workflows from 253 repositories across four frameworks, it yields a deflationary but genuine finding: real workflows are mostly small static pipelines, and only 2 of 187 flagged defects are real bugs, the rest extraction artifacts or intentional design. Curated-benchmark defect rates do not stand in for prevalence.

The contribution survives the rarity of defects. First, a measurement instrument and the first real-world base rates for agent-workflow topology defects. Second, a quantified diagnosis: extraction fidelity, not the check algorithms, is the binding constraint—edge recall 0.64, collapsing to 0.37 on implicit-topology frameworks like ADK, with every triaged structural flag an extraction artifact. Third, a defect-independent use for the static machinery: the graph $\times$ DFA product soundly prunes provably-inert runtime monitors, making pre-deployment analysis a monitor-selection layer within a static-plus-runtime safety stack. We read the negative prevalence result not as a null result but as a redirection—toward higher-fidelity extraction and risk-aware policy specification, the direction we pursue in follow-up work on runtime shields.

Artifact. The tool, corpora, mining and validation pipeline, and all evaluation scripts will be released under the MIT license.

References

[1] CrewAI documentation. `https://docs.crewai.com`, 2025. Accessed: 2026-02-28.

[2] Google Agent Development Kit (ADK) documentation. `https://google.github.io/adk-docs/`, 2025. Accessed: 2026-02-28.

[3] LangGraph documentation. `https://docs.langchain.com/oss/python/langgraph/overview`, 2025. Accessed: 2026-02-28.

[4] Anthropic. Anthropic's responsible scaling policy. `https://www.anthropic.com/responsible-scaling-policy`, 2023. Accessed: 2026-03-01.

[5] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), 2011.

[6] Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, et al. NuSMV 2: An opensource tool for symbolic model checking. In *International Conference on Computer Aided Verification (CAV)*. Springer, 2002.

[7] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.

[8] Remco M. Dijkman, Marlon Dumas, and Chun Ouyang. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12): 1281–1294, 2008.

[9] Iason Gabriel et al. The ethics of advanced AI assistants. *arXiv preprint arXiv:2404.16244*, 2024.

[10] Guardrails AI. Guardrails AI: Adding guardrails to large language models. `https://github.com/guardrails-ai/guardrails`, 2023. Accessed: 2026-03-01.

[11] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.

[12] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, et al. LlamaGuard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.

[13] Dongyun Jin, Patrick O'Neil Meredith, Choonghwan Lee, and Grigore Roşu. JavaMOP: Efficient parametric runtime monitoring framework. In *International Conference on Software Engineering (ICSE)*, 2012.

[14] Daniel Kroening and Michael Tautschnig. CBMC – C bounded model checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2014.

[15] METR. METR: Model evaluation and threat research. `https://metr.org`, 2024. Accessed: 2026-03-01.

[16] NVIDIA. NeMo Guardrails: A toolkit for controllable and safe LLM applications. https://github.com/NVIDIA/NeMo-Guardrails, 2023. Accessed: 2026-03-01.

[17] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977.

[18] Protectai. Rebuff: Self-hardening prompt injection detector. https://github.com/protectai/rebuff, 2023. Accessed: 2026-03-01.

[19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[20] Wil M. P. van der Aalst. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.

[21] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.

[22] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.